

全球首个 · 原生移植技术记录

《杀戮尖塔 2》 iOS 原生移植

Slay the Spire 2 · Native iOS Port

真原生运行 · 非串流 · 非模拟器

技术方案 · 踩坑实录 · 复现指南

2026 年 7 月

仅供技术研究。需使用者自己拥有 Steam 正版，用其正版游戏文件在自有设备上运行，
不涉及盗版分发。

一、TL;DR：做成了什么

把 PC 上的《杀戮尖塔 2》(Godot 引擎 + C#) 编译成 iPhone 原生 App，真机流畅运行、存档与电脑互通，并额外实现了一个连 Steam Mod 都无法上手机的「一键重开本房间」功能。核心难点是绕过 iOS「禁止运行时生成代码」的限制，以及突破单应用内存上限。

能力	状态	关键手段
iPhone 原生运行游戏	✓	官方 Godot 4.5.1 引擎 + NativeAOT 预编译游戏程序集
手机适配补丁（触屏/UI/生命周期）	✓	Mono.Cecil 静态 IL 织入（替代 Harmony 运行时补丁）
音频(FMOD)/动画(Spine)	✓	自行编译两个 GDExtension 的 iOS 版
存档电脑↔手机互通	✓	Steam 存档 → Xcode 容器替换
流畅无卡顿	✓	着色器预热 + 突破内存上限
独家：一键重开本房间	✓	暂停菜单注入按钮 + 复用游戏原生重载逻辑

二、为什么这个游戏能移植

并非所有 PC 游戏都能上手机，这一个能，靠三个前提：

- **引擎选对了**：尖塔2 从 Unity 换成了开源的 **Godot 引擎** (4.5.1)，Godot 官方支持导出 iOS，给移植留了口子。
- **数据包未加密**：游戏资源包（**.pck**）是标准 GDPC 格式、未加密，可解析、可重打包。
- **有安卓先例参考**：社区已有非官方安卓启动器，其手机适配补丁（触屏、UI 缩放）逻辑可移植——但它用的运行时改代码方式在 iOS 行不通，需换实现。

三、核心技术方案

1. 绕过 iOS 的 JIT 禁令 —— 静态 IL 织入

安卓版靠 **Harmony** 在运行时动态给游戏打补丁。但 iOS **禁止程序运行时生成/修改机器码**（无 JIT），Harmony 直接不可用。

解法：写一个基于 **Mono.Cecil** 的「织入器」，在编译阶段把补丁的 prefix/postfix 钩子**静态焊进**游戏主程序集 **sts2.dll** ——运行时不再需要任何动态改代码。安卓的 16 个补丁里，除 1 个纯调试日志（弃用）外全部改写为静态织入。

```
# 织入：把补丁钩子焊进游戏 dll
STS2Weaver sts2.dll 补丁库.dll manifest.json → 织入版 sts2.dll
# manifest 每条 = 目标类.方法 + prefix/postfix + 钩子
```

2. 用苹果预编译器把游戏编成原生库 —— NativeAOT

iOS 上 Godot 的 C# 只有一条路：**NativeAOT**（提前编译，把整个游戏 + .NET 运行时编成一个原生 **.dylib**）。我们没有游戏源码，只有编译好的 **sts2.dll**，于是用一个「空壳工程」——不含源码，在预编译器读取前用织入版 dll 覆盖中间产物，让它把我们的版本编成原生库。

✓ 最大风险点已排除

游戏靠「反射」注册 1624 个卡牌/怪物类型，反射与预编译天生冲突（很多 C# 游戏移植 iOS 死在这）。实测编译通过、运行正常。

3. 音频与动画插件的 iOS 版

游戏依赖 FMOD（音频）和 Spine（动画）两个 GDExtension，官方数据包里只有桌面版库。我们按游戏所用版本（FMOD 2.03.14、Spine 4.2）自行编译出 iOS arm64 版，入口符号与游戏要求逐一核对匹配。

4. 数据包的两处手术

MegaCrit 私有引擎打的 **.pck** 与官方引擎的机制有两处冲突，必须改：

- **移除 Sentry**：崩溃上报插件无 iOS 版，启动加载即崩，从包内移除。
- **补 672 个脚本占位**：场景用路径引用 C# 脚本（**res://...cs**），但包内 0 个 .cs 文件——官方引擎按路径找不到就崩。补上占位文件后，引擎经脚本属性映射到预编译类型即可。

四、踩坑实录（精选）

真机首次运行是一场「层层剥壳」，每个报错都靠日志/崩溃报告定位根因，而非猜测。

现象	根因	解法
编译期 IntPtr 重复定义	误把游戏自带的整套 .NET 运行时库当依赖，与预编译器自带的撞车	以框架实际安装目录为准，精确滤除框架库
预编译器拒载 Steamworks 某类型	一个 C 联合体结构（引用与数字叠在同一内存），严格模式不接受	不强制加载全部依赖，让裁剪丢弃不可达类型
卡在 logo：hostfxr 加载失败	游戏主库没被做成 framework 嵌入 app	从预编译产物造 framework 放入 app
致命错误弹窗（图集加载）	预编译默认禁用反射式 JSON 解析	csproj 开启 <code>JsonSerializerIsReflectionEnabledByDefault</code>
加载场景脚本崩溃	包内缺 .cs 脚本资源（见上）	补 672 个占位脚本
FMOD 初始化 调空地址崩	静态链接缺 <code>load_all_fmmod_plugins</code> 符号	注入零插件空实现
ModelDb 初始化卡死	为绕桌面 bug 写的反射补丁在预编译下卡住——但该 bug 在 iOS 根本不存在	移除该补丁，用游戏原版逻辑（更干净）

△ 最硬的一关：三个毛病、同一个根

「保存并退出闪退」「着色器预热崩」「偶尔卡顿」看似三个问题，从系统的 `JetsamEvent` 日志（`largestProcess: StS2, reason: per-process-limit`）确认**同一个根因：游戏内存超过 iOS 单应用上限被系统强杀**。一个内存权限 `increased-memory-limit`（把上限提到设备内存的 ~75%）同时治好三者。该权限免费账号可用，但需经 Xcode 生成含此权限的配置文件才能签入。

五、独家功能：一键重开本房间

对应 Steam Mod 「Quick Restart 2」，但 Mod 依赖游戏的 Mod 加载器 + 运行时改代码 + 创意工坊订阅，全部上不了手机。功能本身很简单，直接原生重写：

- **加按钮**：静态织入 hook 暂停菜单初始化，复制现有「保存并退出」按钮（样式布局自动一致），改名「重开本房间」，放在「继续」下方。
- **重载逻辑**：复用游戏自己的「退主菜单→继续」路径，但不存档——磁盘上进房间时的存档原封不动，于是等效「撤销这一房间的操作、重来」。全程黑屏过渡，不闪主菜单。

六、着色器预热（消除卡顿）

Godot 手机端通病：首次遇到新画面时现场编译着色器，卡一下。移植安卓版的预热逻辑（首次启动把所有材质在隐藏视口渲染一遍、全部编译并缓存、写完成标记），之后每次启动跳过、全程流畅。注意：预热额外吃内存，必须在内存权限到位后才会不会 OOM。

七、复现与维护

△ 前置声明：本章是命令级流程，但不是零起点脚本

真正跑通还依赖本项目自研的源码资产：**Cecil 织入器 (STS2Weaver)**、**11+ 个移植补丁**、**pck 处理工具**、**导出工程配置 (build-ios.sh 等)**。这些不在本文档内。下述命令用于说明每一步在做什么、参数与版本，照抄命令本身不足以复现——需连同上述源码一起获取。

7.1 环境与版本锁定（版本错配是主要坑源）

组件	版本	说明
Godot (mono 版)	4.5.1-stable	必须与游戏所用引擎版本完全一致
.NET SDK	9.0.x	游戏 TFM 为 net9.0；含 ILCompiler 9.0.x
FMOD Engine	2.03.14 (iOS)	与游戏内 bank 格式匹配；需登录 fmod.com 下载
Spine Runtimes	4.2 分支	与游戏 .spskel 版本匹配
Xcode / iOS SDK	Xcode 26.x	提供 NativeAOT 交叉编译的 clang/链接器

7.2 前提

- 自有 Steam 正版《杀戮尖塔 2》（用其 macOS 版游戏文件，不涉及盗版）
- 一台 Mac + Xcode + 一部 iPhone + 一个 Apple ID（免费即可）
- 工具链：`brew install scons xcodegen ios-deploy libimobiledevice`；.NET SDK 从 dot.net 装

7.3 分步流程（build-ios.sh 内部逻辑）

```
# ① 编译补丁库 + 静态织入到游戏 dll
dotnet build src/STS2MobileIos -c Release -o /tmp/patch
dotnet run --project src/STS2Weaver -c Release -- \
  game-refs/sts2.dll /tmp/patch/STS2MobileIos.dll \
  src/STS2MobileIos/manifest.json /tmp/sts2-woven.dll
# ② 组装依赖(以 .NET 共享框架目录为准滤除框架库,只留第三方) → prebuilt/
# ③ 编译 GDExtension: FMOD + Spine 的 iOS arm64 版
scons platform=ios arch=arm64 target=template_release fmod_lib_dir=... # FMOD
./build-extension.sh ios arm64 # Spine(先 setup-extension.sh 4.5.1-stable)
# ④ NativeAOT: 预编译织入版游戏 → 原生 dylib(~75MB arm64)
dotnet publish ios-export/sts2.csproj -c ExportRelease -r ios-arm64 \
  -p:GodotTargetPlatform=ios --self-contained
# ⑤ Godot 导出 Xcode 工程 + 数据包三连(移 sentry + 补 672 脚本占位)
# ⑥ dummy.cpp 注入 FMOD 空实现 load_all_fmod_plugins
```

7.4 签名装机（免费账号关键：内存权限只能经 Xcode 签入）

```
# 首次 / 加内存权限：必须 Xcode GUI(命令行 xcodebuild 报 No Accounts)
# Xcode 打开 build/StS2.xcodeproj → Signing&Capabilities
# → Team 选自己 → +Capability → Increased Memory Limit → Run
# 此步让 Apple 生成含该权限的 provisioning profile 并装机

# 之后增量(仅换补丁 dylib)：命令行即可(profile 已含权限)
cp publish/sts2.dylib "$APP/Frameworks/sts2.framework/sts2"
codesign -f -s "Apple Development: 你的账号" --generate-entitlement-der \
  --entitlements ent_mem.plist "$APP" # ent 含 increased-memory-limit
xcrun devicectl device install app --device <UDID> "$APP" # 覆盖,勿卸载
```

⚠ 三条铁律

- 覆盖安装，绝不卸载重装——卸载会清空存档。
- 免费签名 7 天失效，Xcode 重新 Run 一次即可续签，存档不受影响。
- 游戏更新后需重跑构建（着色器缓存会重置，首次启动重新预热一次）。

存档迁移

电脑 Steam 存档位于 `userdata/<id>/2868840/remote/`；用 Xcode 「Devices and Simulators → Download/Replace Container」把存档合并进手机 app 的 `Documents/default/1/` 即可。